Informatik 12
Cauerstraße 11
91058 Erlangen

FRIEDRICH-ALEXANDER
UNIVERSITÄT
ERLANGEN-NÜRNBERG
TECHNISCHE FAKULTÄT

Embedded Systems
2. Extended Exercise

Contents

1 Introduction **2**

2 Preparation **2**

2.1 Fundamentals of Architectural Synthesis 2

2.2 Architectural Synthesis using Vivado HLS 2

3 Experiment **3**

3.1 Preparation 3

3.2 Input and Output Protocoll 4

3.3 Differential Equation Solver 4

3.4 Architectural Synthesis 5

3.5 Adjustment of the Fixed Point Representation 7

3.6 Logic Synthesis 7

A References **7**

B EquationSolver.cc **8**

C EquationSolver.h **9**

D datatypes.h **10**

E Handshake Protocol **11**

1 Introduction

In the lecture Embedded Systems, architectural snythesis (or high-level synthesis) is defined as the process of generating a structural register transfer level description from a given behavioral description. The main task of the synthesis is the *allocation* of hardware units, the *binding* to allocated hardware units and of variables to registers and the *scheduling* of all tasks on the hardware units.

In the first extended exercise, these tasks were done by hand for the differential equation solver. In this extended exercise, a commercial tool (Vivado HLS from Xilinx [2]) is used to perform this synthesis semi automatically. Vivado HLS supports both SystemC RTL as well as Verilog RTL as output format. Such an RTL description kann be refined by other tools (for logic synthesis, place and route, etc.) for different plattforms (e.g. FPGA, ASIC).

2 Preparation

The tools and libraries used in this experiment run under Linux on the Chair's workstations. We therefore assume that you are at least familiar with the basic principles of this operating system.

Please prepare for the exercise, by reading the script, understanding the basics (will be queried), preparing Chapter 1 at home, and reflecting the tasks of the two chapters, otherwise experience shows that time quickly runs out. Furthermore, you should prepare the topic of architectural synthesis [4, 2].

2.1 Fundamentals of Architectural Synthesis

Architectural synthesis is the automatic transformation of an algorithmic description into a hardware structure, implementing this behavior. The input of this architectural synthesis is an algorithm, e.g. written in *C, C++* or *SystemC*, as well as a *resource library* for the target plattform (e.g. FPGA, ASIC) and *constraints for the implementation*. The output is an RTL description, often in the form of code in one of both hardware description languages (HDL) *VHDL* or Verilog.

The synthesis tool analyzes the algorithmic description and identifies possible mappings to different resource types from the library. In the next step, the *Allocation*, the number of resources required is determined. The allocation can be influenced by setting different optimization objectives (area, throughput, latency, clock frequency, etc.). Next follows the *binding* of the operations to the resources as well as the *scheduling*. After the optimization the data path with the functional units, registers, and multiplexers is generated. The control path (control unit) implements the scheduling with the corresponding driving of the multiplexers, registers, and functional units (see fig. 1). Finally, the circuit is written to a file in the appropriate output format.

2.2 Architectural Synthesis using Vivado HLS

The architecture synthesis tool *Vivado HLS* expects as input a program in *SystemC*. The SystemC code can be enriched by special macros, which influence the synthesis. This allows e.g. to realize fixed input/output protocols. In the first step the architecture synthesis is accomplished where the output is a VHDL code. In the second step the *logic synthesis* is performed resulting in a description of the circuit in FPGA primitive cells (flip-flops, lookup tables).

A Vivado HLS project (*project.tcl*) consists of the specification of the functional input files and the test bench (tb.cc), the used resource library, as well as several *configurations*. A configuration consists of the synthesis settings and a simulation environment to simulate the synthesized structural

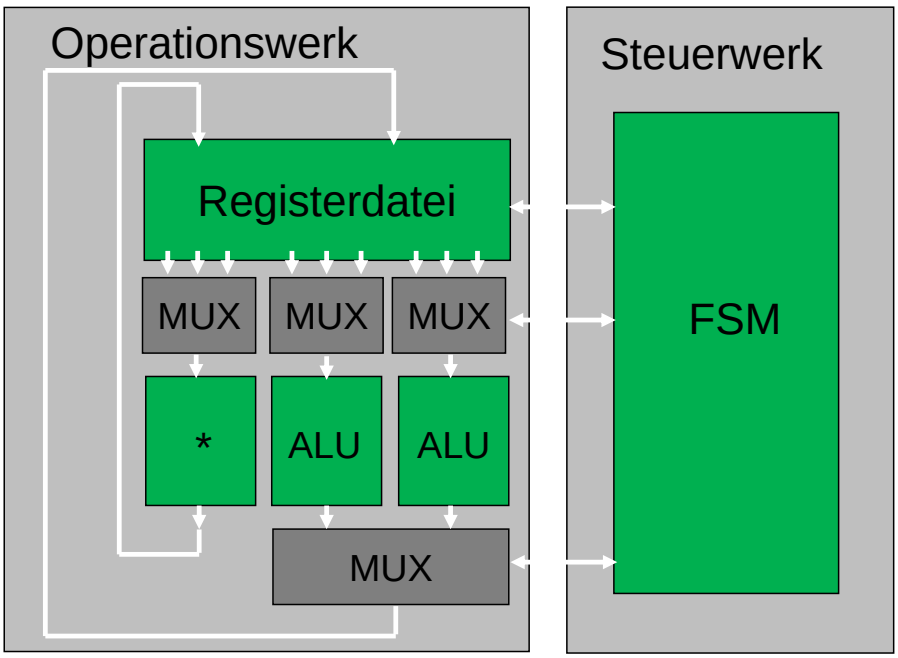


Figure 1: The data path (operation unit) and control path (control unit) consisting of the allocated functional units, muliplexers, and registers. The control unit is realized as a finite state machine (FSM).

SystemC code using the test bench. Furthermore, it is also possible to simulate the functional source code. It is important that the same test bench is used for all simulations for the simulation result to be comparable. As resource library we use an example FPGA library from Vivado HLS.

3 Experiment

The aim of this experiment is the synthesis of the differential equation solver from the first practical course using the synthesis tool Vivado HLS from Xilinx [2]. For this experiment, the differential equation solver (EquationSolver.cc) from the first practical course must first be extended by an input and output protocol for communication with the test bench (tb.cc). A test bench for verification is already available. Furthermore, the multiplication of the fixed-point numbers must be adjusted before synthesis. Then the synthesis can be carried out and various optimization strategies (latency, throughput, costs, etc.) can be carried out. The accuracy of the differential equation solver is then examined by adjusting the fixed-point representation. The synthesized circuits can then be mapped by logic synthesis to *FPGA primitive cells* (flip-flops, lookup tables).

3.1 Preparation

To perform this experiment, you need the Vivado HLS software tool. You will also need an editor (e.g. kate, emacs) to edit the source files. The source files are located in the *Home* directory of your account.

To use Vivado HLS, you have to set up its environment once in the console (shell):

```
module load vivado/2019.2_x86_64-pc-linux
```

The Vivado HLS documentation can be found in your project directory. Further information can be found at [3].

Change to the prepared working directory, where you will find the source files for the differential equation solver:

```
cd Erweiterte_Uebung2/versuch
```

The project directory contains the source files for the differential equation solver. Open an editor of your choice and then the files *EquationSolver.cc* and *datatypes.h* from the directory *~/Erweiterte_Uebung2/versuch*.

First look at the contents of the file *EquationSolver.cc* (see appendix B). In the further course of the practical course you will first modify the methods *dataIn()* and *dataOut()*. These methods shall implement the protocol for input and output of data to the circuit and are used by the SystemC process *thread0* for communication with the test bench. In this process you already find the source code of the differential equation solver from the last practical course. However, this code is still deactivated by the preprocessor macro *TEST_PROTOCOL* and instead a simple substitute function is calculated. This substitute function shall first be used to implement and test the input and output protocol. The differential equation solver can then be prepared for synthesis with Vivado HLS. During the actual synthesis, you can try different optimizations and compare the synthesis results with each other. Finally, the influence of different fixed point representations on the quality of the algorithm will be investigated.

3.2 Input and Output Protocoll

First implement the input protocol in the *dataIn()* function. In the appendix E, the use of such protocols is described. For the communication of the equation solver with the test bench it is best to use a *Handshake* protocol as described on page 58 or in the example on page 70 (see appendix).

The *Handshake* protocol for the input looks roughly as follows: The signal *ready_in* indicates that input data can be received. Then wait until the input data is valid (signal *valid_in*). Then you can read the input data (x,u,y,a,dx) from the signals (*x0_in, u0_in, y0_in, a_in, dx_in*). The reception of the data is then acknowledged by setting the *ready_in* to zero (*false*) for at least one clock.

Implement the output protocol to write the value of the variable y to the output signal *y_out* analogously.

Save the file to test the protocol. Then test your implementation with the script *handshake.sh*. Compare your result with the given solution.

3.3 Differential Equation Solver

In this task, the behavioral description of the differential equation solver for architectural synthesis in SystemC shall be adapted with the tool Vivado HLS. First activate the source code of the equation solver by deactivating the definition of the preprocessor macro *TEST_PROTOCOL*. If you now compile and simulate the project (using the script *handshake.sh*) you will notice that your result differs from the given one. The reason lies in the multiplication of fixed point numbers used: If you multiply fixed-point numbers, the number of bits required for a lossless calculation increases. For example, after multiplying two 32-bit fixed-point numbers, the result is a 64-bit fixed-point number. This can mean a very large resource expenditure when concatenating multiplications (word width is doubled in each case). However, only 32-bit fixed-point numbers should be used in the given result.

Modify the equation solver such that only 32-bit numbers are used for calculation. This can be achieved by simple C/C++ *cast* operations. The data type for the 32-bit fixed-point number is *eq_fixed_t*.

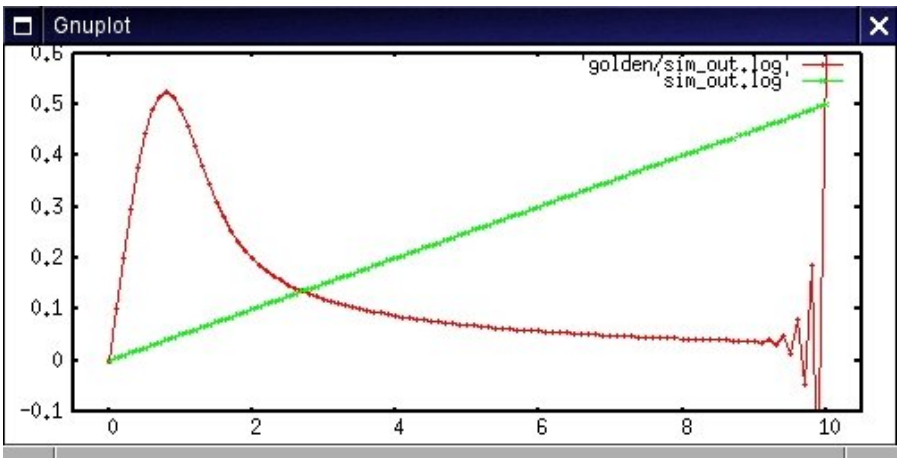


Figure 2: Plot of the output after completion of the input and output protocol. The substitute function for testing the input and output protocol is shown in green.

Can you guess what happens with this measure? For example, what do you have to consider when converting?

By reducing the multiplication results to 32 bits, it is important to parenthesize the multiplications. A changed order of execution would cause a slightly different output due to *rounding error*. To get comparable results, use the multiplication order shown in the figure below. 3 of the differential equation solver. For the constant 3 (*const 3*) a signal should be used that returns a constant 3 (*const3_in.read()*). For Vivado to recognize the pragmas, the already inserted identifier *datapath_loop* must be used.

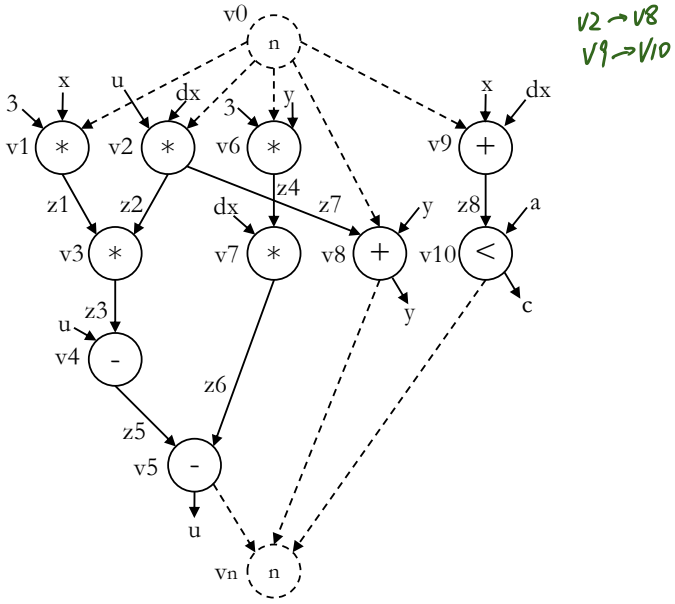


Figure 3: Data-flow graph

3.4 Architectural Synthesis

Next, different configurations of the Vivdao HLS tool will be investigated. Among other things, latency-optimal solutions as well as resource-limited solutions are to be determined. The so-called constraints are identified by the command *set directive*. Execute the following command:

```
vivado_hls -f script.tcl
```

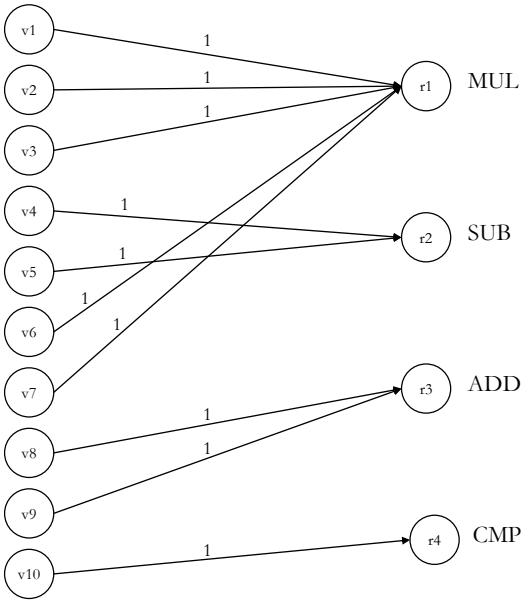


Figure 4: Resource graph

Then you can start the GUI of Vivado HLS for your created project:

```
vivado_hls -p Vivado_RTL/
```

Now open the file *script.tcl*. The script contains four different configurations with different optimization goals for architecture synthesis. You can find a documentation of the different targets under [1]. The script contains the following configurations:

- *project_bare*: Latency minimization with dedicated resources without functional pipelining (without iterative scheduling)
- *project_pipelined*: Functional pipelining with specification of the required iteration interval (period) $\Pi = 1$.
- *project_pipelined_faster_clock*: Functional pipelining with specification of desired iteration interval (period) $\Pi = 1$ and additional specification of takt period (ns)
- *project_area_mapping_111*: Configuration with a fixed allocation:
  $\alpha = \{1\text{xMUL}, 1\text{xADD}, 1\text{xSUB}, 1\text{xCMP}\}$ (as in Fig.4)

Task:

- 1. Explore Pareto-optimal allocations for area costs and latency. Advice: Successively modify the configuration *project_area_mapping_111*.
- 2: Determine a resource-minimal allocation that can achieve a latency of five (L=5).

Capture your results (latency and resource allocation) in a table and plot them in a diagram. For example, for this you can use LibreOffice Calc. Highlight the Pareto-optimal points in your graph.

3.5 Adjustment of the Fixed Point Representation

Next, you will examine the influence of the fixed-point representation on the accuracy of the implementation of the differential equation solver. The accuracy is defined in the file *datatypes.h*. This file contains the declaration for the (already used) data type *eq_fixed_t*. The default data type is *sc_fixed<32, 16>* Type with 32-bit fixed-point number, where 16-bit is used for the integer part.

Modify this declaration to test fixed-point representations with different numbers of bits for the integer and decimal parts, e.g. *sc_fixed<32, 20>* or *sc_fixed<32, 8>*. Compare the results using plots.

3.6 Logic Synthesis

During logic synthesis, the combinatorial logic of the VHDL modules is optimized and mapped together with the registers to *primitive cells* of a certain target plattform. We use the logic synthesis tool *Vivado* from Xilinx and the FPGA technology *Zynq-7000*.

Start the *Xilinx Integrated Development Environment Vivado* on *your* computer as follows:

```
vivado
```

Then open a new project under the path for each configuration:

```
<project directory>/<configuration>/impl/vhdl/project.xpr
```

Then click on the *Elaborated Design* tab. Look at the RTL design (tab *Schematic*) and see which elements the circuit is made of and what they are needed for. Look for (1) the controller and (2) check if and how the synthesis constraints affect the RTL design. Is the existing control unit microprogrammed?

A References

References

[1] Vivado High Level Synthesis. https://www.xilinx.com/html_docs/xilinx2017_2/sdaccel_doc/topics/pragmas/concept-intro_to_hls_pragmas.html.

[2] Vivado High Level Synthesis. https://www.xilinx.com/products/design-tools/vivado/integration/esl-design.html.

[3] Vivado High Level Synthesis. https://www.xilinx.com/support/documentation-navigation/design-hubs/dh0012-vivado-high-level-synthesis-hub.htm.

[4] J. Teich and C. Haubelt. *Digitale Hardware/Software-Systeme: Synthese und Optimierung*. Springer, 2007.

B EquationSolver.cc

```
#include "EquationSolver.h"

//////////////////////////////////////////////////////////////////////
// EquationSolver.cc - single thread SC_MODULE
//////////////////////////////////////////////////////////////////////

//
// reset() - EquationSolver reset code
//
// Every SC_CTHREAD to be synthesized must implement reset using a
// global reset_signal_is statement (in the constructor). The reset
// functionality itself consists of a single CYN_PROTOCOL block that
// contains a single wait() call at the end. This code can be placed
// in a reset function, which is called at the top of thread0(). All
// functionality within this protocol block will appear in the reset
// state of the FSM created by Cynthesizer.
//
void EquationSolver::reset()
{
  {
#pragma HLS protocol floating
    // Put reset code here
    // Make sure to initialize all output ports
    // and all data members of the module.

    // You should not check for the reset signal itself being active
    // here (e.g. do not use "if (rst == 0)" ). Putting a conditional
    // check in the reset code will generate a Cynthesizer warning
    // because the RTL output may violate RTL coding guidelines. The
    // reset_signal_is mechanism of SystemC ensures that code before
    // the first wait() statement is executed every time reset is
    // active, so you do not need to check for the reset being active.

    // The reset block should end with a wait() call (this is the
    // first wait() in the thread).

    y_out=0;
    valid_out = 0;

    wait();
  }
}

//
// dataIn() - input hardware interface
//
// This function represents the input interface of the SC_MODULE.
// Place code within the CYN_PROTOCOL block of this function that
// will implement the protocol and timing necessary to read data
// into the design.
//
void EquationSolver::dataIn()
{
  {
#pragma HLS protocol floating
    // Put input interface code here

    // Aufgabe 3.2a)
    // Hier muss das Handshake-Eingabeprotokoll implementiert werden.
    // Ziel ist es, die Variablen x, u, y, a und dx von den jeweiligen
    // Ports x0_in, u0_in, y0_in, a_in, dx_in zu lesen. Mit dem Signal
    // ready_in signalisiert man der Testbench, dass man Daten lesen
    // moechte. Mit dem Signal valid_in signalisiert die Testbench diesem
    // Modul, dass die Daten jetzt gelesen werden koennen.

  }
}

//
// dataOut() - output hardware interface
//
// This function represents the output interface of the SC_MODULE.
// Place code within the CYN_PROTOCOL block of this function that
```

```
// will implement the protocol and timing necessary to output data
// from the design.
//
void EquationSolver::dataOut()
{
  {
#pragma HLS protocol floating
    // Put output interface code here

    // Aufgabe 3.2b)
    // Hier muss das Handshake-Ausgabeprotokoll implementiert werden.
    // Ziel ist es, die Variable  y, auf den jeweiligen Ausgangsport
    // y_out zu schreiben. Mit dem Signal
    // ready_out signalisiert die Testbench, dass sie Daten lesen
    // moechte. Mit dem Signal valid_out signalisiert man der Testbench
    // dass die Daten jetzt gelesen werden koennen.

  }
}

//
// thread0() - main module thread
//
// All of the functionality of the single SC_CTHREAD design should
// be placed within this thread. The functionality can consist of as many
// method calls with argument passing as desired.
//
void EquationSolver::thread0()
{
  // Call the reset function. This must be done at the top of the
  // thread for correct reset handling. It is not necessary to define
  // a separate reset function, but the reset functionality must
  // appear as a single CYN_PROTOCOL block with a single wait() at the
  // end, and it must be the first executable code in the thread's
  // body.

  reset();

  while( true ) {
    dataIn();
    //-----------
    // Aufgabe 3.3
    // -----------
    // Der Differenzialgleichungsloeser aus dem letzten Praktikum.
    // Dieser Code muss noch angepasst werden.
    // Zum Testen muss aber zuerst das Ein-/Ausgabeprotokoll implementiert werden.
#define TEST_PROTOCOL
#ifdef TEST_PROTOCOL
    y = eq_fixed_t(0.05)*a;
#else // TEST_PROTOCOL
    while(x < a) {
      x1 = x + dx;
      u1 = u - ( eq_fixed_t(3) *x * u * dx ) - ( eq_fixed_t(3) * y * dx );
      y1 = y + (u * dx);

      x = x1;
      u = u1;
      y = y1;
    }
#endif // TEST_PROTOCOL

    dataOut();
  }
}
```

C EquationSolver.h

```
#ifndef EQSOLV_H_INCLUDED
#define EQSOLV_H_INCLUDED

#include "datatypes.h"

SC_MODULE(EquationSolver)
{
```

```
public:
  sc_core::sc_in< bool >         clk;
  sc_core::sc_in< bool >         rst;
  sc_core::sc_in< bool >         valid_in;
  sc_core::sc_out< bool >        ready_in;
  sc_core::sc_in< eq_fixed_t >   x0_in;
  sc_core::sc_in< eq_fixed_t >   u0_in;
  sc_core::sc_in< eq_fixed_t >   y0_in;
  sc_core::sc_out< bool >        valid_out;
  sc_core::sc_in< bool >         ready_out;
  sc_core::sc_out< eq_fixed_t >  y_out;
  sc_core::sc_in< eq_fixed_t >   dx_in;
  sc_core::sc_in< eq_fixed_t >   a_in;
  sc_core::sc_in< eq_fixed_t >   const3_in;



  SC_CTOR(EquationSolver) {
    SC_CTHREAD(thread0, clk.pos());
    reset_signal_is(rst, false);

    // don't initialize data members or output ports here
    // in the constructor. do it in the reset() function instead.
  }

 private:

  void thread0();
  void reset();
  void dataIn();
  void dataOut();

  // place any additional functions or data members here
  //   eq_fixed_t x;
  //   eq_fixed_t u;
  //eq_fixed_t y;
  //eq_fixed_t x1;
  //eq_fixed_t u1;
  //eq_fixed_t y1;
/*    double x; */
/*    double u; */
/*    double y; */
/*    double x1; */
/*    double u1; */
/*    double y1; */
/*    sc_int<32> x; */
/*    sc_int<32> u; */
/*    sc_int<32> y; */
/*    sc_int<32> x1; */
/*    sc_int<32> u1; */
/*    sc_int<32> y1; */

  eq_fixed_t x;
  eq_fixed_t u;
  eq_fixed_t y;
  eq_fixed_t x1;
  eq_fixed_t u1;
  eq_fixed_t y1;
  eq_fixed_t a;
  eq_fixed_t dx;

  //eq_fixed_t yout;
  //eq_fixed_t c3;
  //eq_fixed_t dx3;


};

#endif // EQSOLV_H_INCLUDED
```

D datatypes.h

```
#ifndef __DATATYPES_H_INCLUDED__
#define __DATATYPES_H_INCLUDED__
```

```
#define SC_INCLUDE_FX
#include <systemc>

//#include <cynw_fixed.h>

typedef sc_dt::sc_fixed<32,16> eq_fixed_t;


#endif // __DATATYPES_H_INCLUDED__
```

E Handshake Protocol

Using handshaking protocols

As pointed out previously, protocol blocks are implemented exactly as they are written, while internal behavioral code can be directed to be implemented in any number of ways. One architecture might implement a multiplication operation in one cycle, and another could save hardware and implement it in eight clock cycles. Because of this, it is difficult to write a good, multipurpose testbench without some knowledge of when the design will be ready to accept new input values or has valid data on its output ports.

In order for a testbench to know when to create input values and when to read the correct output values, some additional ports must be added. For each input port, an "input ready" output port must be added to tell the testbench that the design is ready for new data. Also, an "input valid" input port must be added so the testbench can tell the design that it can begin to read the values on the port.

Conversely, each output port also needs two ports added: an "output valid" output port, telling the testbench that valid data is on the port, and an "output ready" input port, telling the design that the testbench is ready to read another value on the port. This is called *ready-valid handshaking*, and once it is added any architecture generated by Cynthesizer can be tested with a single testbench. This greatly enhances verification and makes it much more efficient. Ready-valid handshaking is recommended if your design is going to be pipelined (in which case the output from the design for a specific input may be produced any number of clock cycles later, dependent on design constraints and synthesis directives).

To add ready-valid handshaking to a design, begin by declaring the ready and valid ports for each input and output. The `mymod SC_MODULE` declaration might look like this:

```
SC_MODULE( mymod ) {
  sc_in< bool >          clk;
  sc_in< bool >          reset;
  sc_in< sc_uint<4> >    data_in;
  sc_out< bool >         data_in_rdy;  // Input ready handshake
  sc_in< bool >          data_in_vld;  // Input valid handshake
  sc_out< sc_uint<8> >   data_out;
  sc_in< bool >          data_out_rdy;  // Output ready handshake
  sc_out< bool >         data_out_vld;  // Output valid handshake
  …
```

```
};
```

Now, all operations on input or output ports must include loops that wait until the ready-valid handshaking conditions are met. The protocol block that reads values through the input port might look like this:

```
{
  CYN_PROTOCOL( "read" );
  sc_uint<6> i = 0;
  do {
    data_in_rdy = 1;
    while (!data_in_vld)
      wait(1);
    BUFx[i] = data_in.read();
    wait(1);
    i++;
  } while( i != 32 );
  data_in_rdy = 0;
}
```

Here the `for` loop is replaced with a `do-while` loop that performs 32 iterations. The loop begins by asserting `data_in_rdy` high. This tells the testbench that the `mymod SC_MODULE` is ready to read new values on the `data_in` input port. The design then sits in a `while` loop waiting until the testbench acknowledges that it has placed a value on the port (by asserting `data_in_vld`). The input port is read, the design waits one cycle as the original protocol stated, the iteration counter is advanced, and the `do-while` loop is repeated. This handshake continues for each input value to be read. When the loop is complete, the `data_in_rdy` output is reset, telling the testbench that the design is busy processing the input values that were just read.

For the output protocol, handshaking ports are introduced into the design like this:

```
{
  CYN_PROTOCOL( "write" );
  sc_uint<6> i = 0;
  do {
    data_out = BUFy[i];
    data_out_vld = 1;
    do {
      wait(1);
    } while(!data_out_rdy);
    i++;
```

```
  while( i != 32 );
  data_out_vld = 0;
}
```

Again, the `for` loop is replaced with a `do-while` loop. The loop begins by making the output assignment. The design asserts `data_out_vld`, telling the testbench that valid data is on the port. The design then waits in a `do-while` loop for the testbench to acknowledge that it is ready to read this output value (by asserting `data_out_rdy`). The iteration counter is incremented and the loop repeats for all 32 output values. When the loop is complete, the design resets `data_out_vld`, telling the testbench that it is done producing values.

Using protocol blocks to register inputs

In addition to establishing a fixed protocol for all input and output interfaces, protocol blocks are also useful for controlling the registering of input signals. The style in which they are written can determine whether an input is registered or not. Consider this protocol block:

```
{
  CYN_PROTOCOL( "io_behv" );
  in1_x = in1.read();
  in2_x = in2.read();
  out1.write( in1_x && in2_x );
  wait();
}
```

Here some input values are read and then immediately used to determine the value of system output `out1`. A `wait()` statement is present afterward, but because of the way the protocol block is written, the input values are not registered when they are used in the design. To register the system inputs, simply move the `wait()` statement up so it appears immediately after the reading of the inputs, as follows:

```
{
  CYN_PROTOCOL( "io_behv" );
  in1_x = in1.read();
  in2_x = in2.read();
  wait();
  out1.write( in1_x && in2_x );
}
```

The presence of the `wait()` statement ensures that the input values are registered before being used elsewhere in the design.

Setting an input delay to allow use of inputs without registers

If Cynthesizer knows how far into each clock cycle the design's inputs can be expected to be stable, it can safely schedule operations that use those inputs during the remainder of the clock cycle. If it does not know the input delay, it must register each input before using since it doesn't know how much time is available in which to schedule operations.

The most common method of specifying an input delay is to define the delay for all input ports in the design using the `CYN_DEFAULT_INPUT_DELAY` directive. If the external module that writes these inputs has registered outputs, then a safe value for the input delay can be derived from the *clk->q* delay for registers in the target library. The following example shows how to use `CYN_DEFAULT_INPUT_DELAY`:

```
SC_MODULE(dut) {
   sc_in_clk clk;
   sc_in<bool> rst;
   sc_in<bool> enable;
   sc_in< sc_uint<16> > in_data;
   sc_in<bool> out_busy;
   sc_out<bool> out_vld;
   sc_out< sc_uint<16> > out_data;


   SC_CTOR(dut) {
      // Declare that all inputs will be stable 0.5 ns after
      // the start of each clock cycle.  This value may be the
      // clk->q delay for the upstream module.
      CYN_DEFAULT_INPUT_DELAY( 0.5, "input delay" );

      SC_CTHREAD(t,clk.pos());
      reset_signal_is( rst, 0 );
   }
   void t()
   {
      while (1) {
         // Read an input.
```

```
        bool was_vld;
        sc_uint<16> din, q;
        {  CYN_PROTOCOL("read");
           wait();
           was_vld = enable.read();
           din = in_data.read();
        }

        // Calculate.
        sc_uint<16> q = f(din);

        // Write an output.
        {  CYN_PROTOCOL("write");
           do { wait(); } while (out_busy.read());
           out_vld = was_vld;
           out_data = q;
        }
    }
};
```

The `CYN_DEFAULT_INPUT_DELAY` directive in the module's constructor will apply to each `SC_CTHREAD` in the module. To affect the timing of inputs in an individual thread, the directive can be placed it the thread's function instead.

Since the timing of each input is known, Cynthesizer will be able to schedule calculations in the `f()` function based on the `in_data` input in the same cycle that the input was read. This can reduce latencies, and eliminate the requirement for an input register.

Timing values for individual inputs can also be specified using the `CYN_INPUT_DELAY` directive. The default delay value may also be specified via the `--default_input_delay` command line option.

Declaring inputs as stable to avoid any registering

It is common for some design inputs to be registered by another module for well defined periods of time. These are called *stable inputs*. Cynthesizer will allow stable inputs to be used in calculations directly without inferring a register. The `CYN_STABLE_INPUT` directive can be used identify stable inputs, and to identify the region in which they can be expected to be stable.

In the following example, inputs `reg1` and `reg2` can be assumed to be held in registers by an external module for the time between the first `CYN_PROTOCOL` block and the last `CYN_PROTOCOL` block:

```
SC_MODULE(dut)
{
   sc_in_clk clk;
   sc_in<bool> rst;
   sc_out<bool> rdy_in;
   sc_in<bool> vld_in;
   sc_in< sc_uint<16> > reg1;
   sc_in< sc_uint<16> > reg2;
   sc_in<bool> rdy_out;
   sc_out<bool> vld_out;
   sc_out< sc_uint<16> > data_out;

   SC_CTOR(dut) {
      CYN_DEFAULT_INPUT_DELAY(0.5,"input delay");
      SC_CTHREAD(t, clk.pos());
      reset_signal_is(rst, 0);
   }
   void t() {
      sc_uint<16> q;
      while (1) {
         // Wait for a valid input.
         {  CYN_PROTOCOL("read");
            do {wait();} while (!vld_in.read());
         }
         {  //Define a block during which reg1 and reg2 are stable.
            CYN_STABLE_INPUT( reg1, "" );
            CYN_STABLE_INPUT( reg2, "" );

            // Calculate the result in this block.
            q = f(reg1.read(),reg2.read());
         }
         // Write the result.
         {  CYN_PROTOCOL("write");
            vld_out = 1;
            data_out = q;
            do {wait();} while (busy_out.read());
            vld_out = 0;
         }
   }
}
```

Notice that the CYN_STABLE_INPUT directive appears at the top of a block delimited by curly braces (e.g. {}). This is a free-scheduling region. The directive implies that the reg1 and reg2 inputs will have a stable value at any time in that block, and that Cynthesizer can schedule reads of those inputs at any time during that block. This will prevent a register from ever being inferred for an input read in a CYN_STABLE_INPUT block.

Defining a CYN_STABLE_INPUT block immediately between two bounding CYN_PROTOCOL blocks is very common because it identifies the time during which the external module has agreed to keep the inputs registered. Notice that the reg1.read() and reg2.read() expressions do not appear in a CYN_PROTOCOL block.

The CYN_STABLE_INPUTS directive is also provided for cases where there are a large number of stable inputs must be identified. It allows you to specify the first and last inputs in a range. Any other inputs falling in that range in the SC_MODULE definition will also be considered stable. In this example CYN_STABLE_INPUTS would be used as follows:

```
{  //Define a block during which reg1 and reg2 are stable.
   CYN_STABLE_INPUTS( reg1, reg2, "" );
```

For more information, see “Specifying input delays” (page 429, *UG*).

Accessing external memories in free-scheduled regions

External memories are accessed through metaport classes that may either be generated by cynthWB, or handwritten in C++. Each memory read or write is scheduled as a fixed length transaction. These memory transactions can appear in either fixed scheduling or free scheduling regions. More often than not, they appear in free scheduling regions, mingled directly with the code of the algorithm.

For example:

```
SC_MODULE(dut)
{
   // Input and output signals.
   sc_in_clk clk;
   sc_in<bool> rst;
```

```
    sc_out<bool> rdy_in;
    sc_in<bool> vld_in;
    sc_in< sc_uint<16> > data_in;
    sc_in<bool> rdy_out;
    sc_out<bool> vld_out;
    sc_out< sc_uint<16> > data_out;

    // External memory port.
    RAM256X16::port<> mem;

    SC_CTOR(dut) {
       CYN_DEFAULT_INPUT_DELAY(0.5,"input delay");
       SC_CTHREAD(t, clk.pos());
       reset_signal_is(rst, 0);
    }
    void t() {
       sc_uint<16> a, q;
       while (1) {
          // Wait for a valid input.
          {  CYN_PROTOCOL("read");
             do {wait();} while (!vld_in.read());
             a = data_in.read()
          }

          // Read 16 values from the memory, starting at 'a'
          // and sum them.
          q = 0;
          for ( int i=0; i<16; i++ ) {
             CYN_UNROLL(ON,"reads");
             q += mem[a+i];
          }
          // Write the result.
          {  CYN_PROTOCOL("write");
             vld_out = 1;
             data_out = q;
             do {wait();} while (busy_out.read());
             vld_out = 0;
          }
    }
  }
```

In this example the `mem[a]` access will result in a fixed-length memory read transaction taking place. Because this transaction takes place in a free-scheduling

region, Cynthesizer can move the memory read. Cynthesizer will obey the following rules when scheduling the memory read:

- The memory read will take place after the address has been calculated, but before its output is used.
- Memory read operations can be overlapped as long as the operations are separated by a number of cycles equal or greater to the defined initiation interval of the memory read operation. Each memory interface defines its initiation interval, but it is commonly 1, in which case, a memory read can be started every cycle.
- The memory read will not be moved into or past a surrounding `CYN_PROTOCOL` block.

In this last limitation, the `CYN_PROTOCOL` blocks at the start and end of the free-scheduling region in which the memory reads appear create *boundaries* for the movement of the memory reads. By default, all `CYN_PROTOCOL` blocks are treated as boundaries. However, this default boundary behavior can be modified as follows:

- A specific point in a `CYN_PROTOCOL` block can be defined to be boundary using the `CYN_BOUNDARY` directive. For more information, see "CYN_BOUNDARY" (page 83, *RG*).
- The `flags` parameter to a `CYN_MEM_READ_TX` or `CYN_MEM_WRITE_TX` directive can have the `CYN_IGNORE_BOUNDARIES` flag added to it to prevent boundaries from limiting the movement of the operation. For more information, see "CYN_MEM_READ_TX / CYN_MEM_WRITE_TX / CYN_PROTOCOL_TX" (page 128, *RG*).

These rules affect both `cynthWB`-generated external memory port classes, and handwritten external memory port classes. For more information on `cynthWB`-generated external memory classes, see "Using external memories" (page 562, *UG*). For more information on defining memory transactions, see "CYN_MEM_READ_TX / CYN_MEM_WRITE_TX / CYN_PROTOCOL_TX" (page 128, *RG*).

Example of design code transformation

This section shows the transformation of an algorithm written in C to an `SC_MODULE` that can be synthesized using Cynthesizer. First, the raw C program is presented. Next, the algorithm is modified to use bit-accurate SystemC data types and to fit into the SystemC simulation environment. Then hardware detail such as I/O protocol is added and it is put into a form that can be synthesized.

Basic C++ algorithm

Following is a C++ program that sends a set of 16 values into a trivial filter function, which in turn returns a weighted average of the last 3 samples.

```
#include <cstdio>

static float shift[3];
static int coefs[3] = { 15,10,5 };
static int sum_of_coefs;
static int initialized = 0;
static int values[16] =
{128,244,13,16,18,41,23,171,128,129,223,41,18,16,17,4 };

int silly_filter_func( float input_data )
{
   if ( !initialized ) {
      shift[2] = shift[1] = shift[0] = input_data;
      sum_of_coefs = coefs[0] + coefs[1] + coefs[2];
      initialized = 1;
   } else {
      shift[2] = shift[1];
      shift[1] = shift[0];
      shift[0] = input_data;
   }
   return ((shift[0]*coefs[0]) + (shift[1]*coefs[1]) +
(shift[2]*coefs[2]) ) / sum_of_coefs;
}

int main( int argc, char* argv[] )
{
   int i;

   for ( i=0; i<15; i++ ) {
      printf( "result = %d\n", silly_filter_func( values[i] ) );
   }
}
```

C++ algorithm using SystemC constructs

Here is the same algorithm embodied in a C++ function using SystemC data. The `int` and `float` data types used in the C function have been changed to SystemC `sc_uints` with appropriate bit widths to achieve bit-accuracy.

```
#include <systemc.h>

static sc_uint<8> shift[3];
static sc_uint<4> coefs[3] = { 15,10,5 };
static sc_uint<6> sum_of_coefs;
static bool       initialized = 0;
static sc_uint<8> values[16] =
{128,244,13,16,18,41,23,171,128,129,223,41,18,16,17,4 };

sc_uint<8> silly_filter_func( sc_uint<8> input_data )
{
  if ( !initialized ) {
    shift[2] = shift[1] = shift[0] = input_data;
    sum_of_coefs = coefs[0] + coefs[1] + coefs[2];
    initialized = 1;
  } else {
    shift[2] = shift[1];
    shift[1] = shift[0];
    shift[0] = input_data;
  }
   return ((shift[0]*coefs[0]) + (shift[1]*coefs[1]) +
(shift[2]*coefs[2]) ) / sum_of_coefs;
}

int sc_main(int argc, char* argv[])
{
   int i;
   for ( i=0; i<15; i++ ) {
      sc_uint<8> result = silly_filter_func( values[i] );
      printf( "result = %d\n", (int)result );
   }
   return 0;
}
```

Behavioral design ready for Cynthesizer

Here is the same functionality embodied in an SC_MODULE properly structured for synthesis by Cynthesizer.

The filter function has been placed into an SC_CTHREAD. The arguments to the function are passed into the module using a ready/valid handshake protocol. A similar protocol is used for passing the result out of the module. All I/O protocol has been placed in a code block delimited by a CYN_PROTOCOL directive.

```
#include <systemc.h>
#include <cynthhl.h>

SC_MODULE(silly_filter)
{
public:
  sc_in<bool>          reset;
  sc_out<bool>         input_ready;
  sc_in<bool>          input_valid;
  sc_in<sc_uint<8> >   input_data;
  sc_out<bool>         output_valid;
  sc_in<bool>          output_ready;
  sc_out<sc_uint<8> >  output_data;

  sc_in<bool>         CLK;

   SC_CTOR( silly_filter )
   {
     SC_CTHREAD(thread0, CLK.pos());
     reset_signal_is(reset, true);
   }

  void thread0();

protected:
   sc_uint<8>shift[3];
   sc_uint<6> sum_of_coefs;
   bool      initialized;
};

void silly_filter::thread0() {
   const sc_uint<4> coefs[3] = {15,10,5};
   sc_uint<8> sample;
```

```
sc_uint<8> result;

// execute reset logic
{  CYN_PROTOCOL( "Reset" );
   input_ready.write(false);
   output_data.write(0);
   output_valid.write(false);
   initialized = false;
   wait();
}
// multi-cycle initialization behavior
sum_of_coefs = coefs[0] + coefs[1] + coefs[2];

// main functionality
while(1) {
   // read input values
   { CYN_PROTOCOL("Read Loop");
      input_ready.write(true);
      do {
         wait();
      } while ( !input_valid.read() );
      sample = input_data.read();
      input_ready.write(false);
      wait();
   }

   // do the computation
   if ( !initialized ) {
      shift[2] = shift[1] = shift[0] = sample;
      initialized = true;
   }
   else {
      shift[2] = shift[1];
      shift[1] = shift[0];
      shift[0] = sample;
   }
   result = ((shift[0]*coefs[0]) + (shift[1]*coefs[1]) +
             (shift[2]*coefs[2]) ) / sum_of_coefs;

   // write output values
   { CYN_PROTOCOL("Write");
      output_valid.write(true);
      output_data.write(result);
      do {
```

```
            wait();
          } while ( !output_ready.read() );
          output_valid.write(false);
        }
      }
    };
```